

گزارش تمرین کامپیوتری **پنجم**

سیگنال ها و سیستم ها

دکتر اخوان

نارین نیکخواه بهرامی

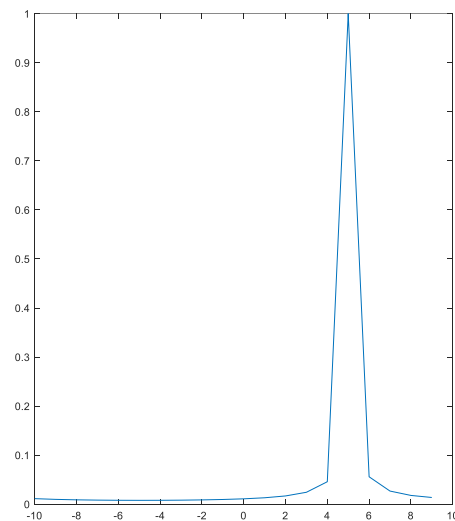
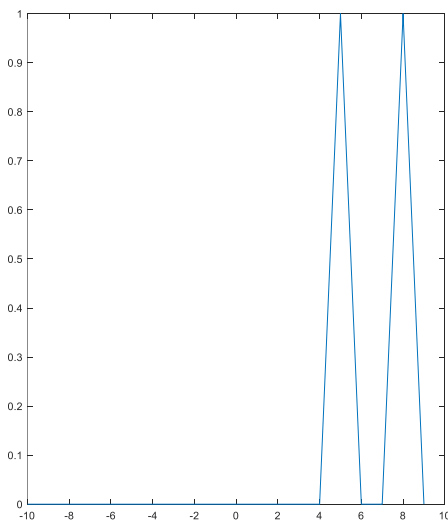
۸۱۰۱۰۲۵۳۰

بهار ۱۴۰۴

تمرین ۱-۰

همان‌طور که مشاهده می‌شود، در نمودار اول، دو سیگنال به‌وضوح از یکدیگر تفکیک شده‌اند. اما در مورد x_2 ، به دلیل این که اختلاف فرکانس سیگنال‌ها کمتر از رزولوشن سیستم است (یعنی 0.1 که کمتر از 1 است)، شناسایی آن‌ها به طور جداگانه ممکن نیست و تنها یک قله در فرکانس 5 هرتز قابل تشخیص است. این نشان می‌دهد که رزولوشن سیستم برای جداسازی فرکانس‌های نزدیک به هم کافی نیست.

```
% Question 0
close all;
clc;
clear;
f=-10:1:9;
fs=20;
ts=1/fs;
t=0:ts:1-ts;
x1=exp(1j*2*pi*5*t)+exp(1j*2*pi*8*t);
x2=exp(1j*2*pi*5*t)+exp(1j*2*pi*5.1*t);
y1=fftshift(fft(x1));
y2=fftshift(fft(x2));
y1=y1/max(abs(y1));
y2=y2/max(abs(y2));
figure;
subplot(1,2,1);
plot(f,abs(y1));
subplot(1,2,2);
plot(f,abs(y2));
```



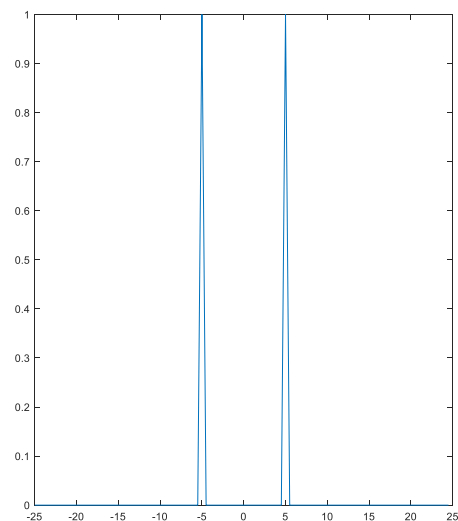
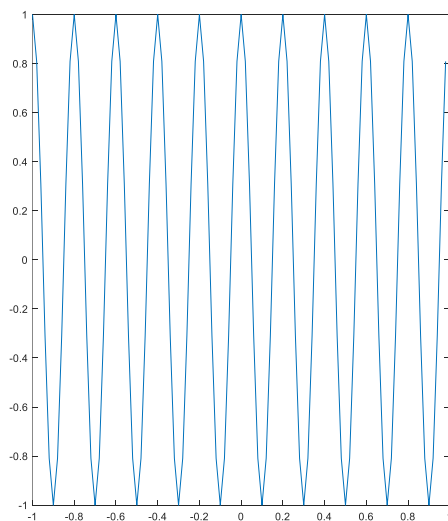
بخش اول:

تمرین ۱-۱

برای تحلیل سیگنال $\cos(10\pi t)$ در حوزه فرکانس، از تبدیل فوریه استفاده می‌کنیم. به این منظور، از تابع `fftshift` برای انتقال سیگنال به مرکز و از `fft` برای انجام تبدیل فوریه بهره می‌بریم. سپس، با تقسیم نتیجه بر بزرگترین مقدار

مطلق آن، طیف را نرمال سازی می کنیم تا ماکزیمم مقدار آن برابر ۱ شود. در پایان، برای نمایش اندازه طیف در حوزه فرکانس، از تابع `abs` استفاده کرده و نمودار ها را رسم می کنیم. مشاهده می کنیم که همان طور که انتظار داریم از تبدیل فوریه یک سیگنال کسینوسی، در فرکانس های ۵ و -۵ اوج های مشخصی وجود دارد. این نشان می دهد که سیگنال ما دارای این مؤلفه های فرکانسی است.

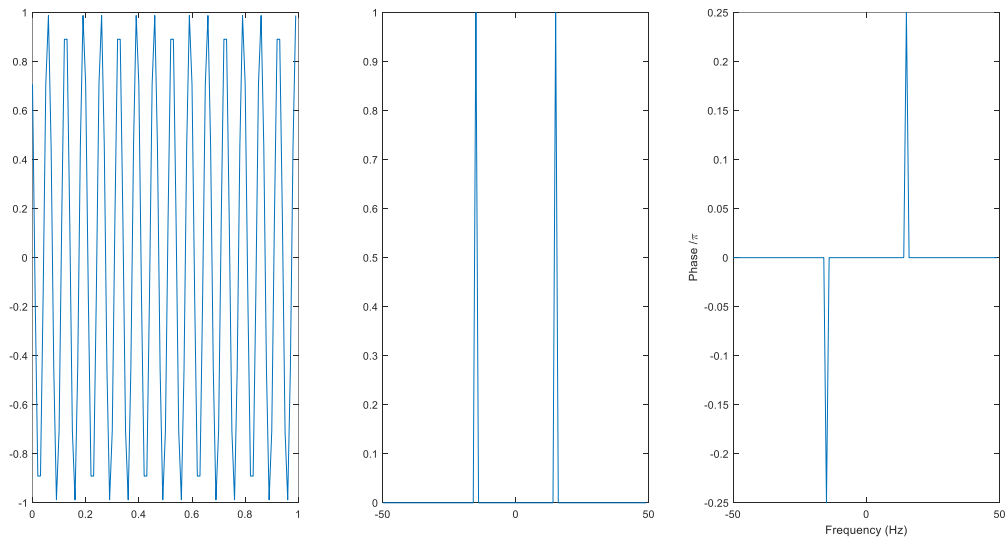
```
% Question 1
close all;
clc;
clear;
%% Part 1
fs=50;
ts=1/fs;
t=-1:ts:1-ts;
N=length(t);
f=-fs/2:fs/N:(fs/2)-(fs/N);
x1=cos(10*pi*t);
figure;
subplot(1,2,1)
plot(t,x1);
y1=fftshift(fft(x1));
y1=y1/max(abs(y1));
subplot(1,2,2)
plot(f,abs(y1));
```



تمرین ۱-۲)

مشابه بخش قبلی، این بار برای سیگنال، مشابه بخش قبل عمل می‌کنیم. همانطور که مشخص است در ۱۵ و -۱۵ مقدار دارد که درست است. درباره‌ی فاز نیز همانطور که انتظار داریم فاز یک سیگنال سینوسی در دو فرکانس با مقادیر متقابل وجود دارد. در این مورد، در فرکانس‌های ۱۵ و -۱۵ فاز ها به ترتیب 0.25 و -0.25 است که نشان دهنده‌ی تقارن فاز در سیگنال های سینوسی است.

```
% Part 2
clear
fs=100;
ts=1/fs;
t=0:ts:1-ts;
N=length(t);
f=-fs/2:fs/N:(fs/2)-(fs/N);
x2=cos(30*pi*t+pi/4);
figure;
subplot(1,3,1)
plot(t,x2);
y2=fftshift(fft(x2));
y2=y2/max(abs(y2));
subplot(1,3,2)
plot(f,abs(y2));
tol=1e-6;
y2(abs(y2)<tol)=0;
theta=angle(y2);
subplot(1,3,3)
plot(f,theta/pi);
xlabel 'Frequency (Hz)';
ylabel 'Phase /\pi'
```



بخش دوم:

تمرین ۲-۱)

یک تابع تشکیل داده ایم که در سطر اول حروف و سطر پایین آن متناظر باینری آن را قرار می‌دهیم.

```
%%Part1 Mapset function
function output = create_mapset()

    Nch = 32;
    output = cell(2,Nch);
    Alphabet = 'abcdefghijklmnopqrstuvwxyz .,;!";';
    for i = 1:Nch
        output{1,i} = Alphabet(i);
        output{2,i} = dec2bin(i-1,5);
    end

end
```

end

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v
2	00000	00001	00010	00011	00100	00101	00110	00111	01000	01001	01010	01011	01100	01101	01110	01111	10000	10001	10010	10011	10100	10101
3																						
4																						
5																						
6																						
7																						
8																						
9																						
10																						
11																						
12																						
13																						
14																						

تمرین ۲-۲ و ۳-۲)

در این تابع ابتدا پیامی که قرار است کدگذاری شود را با استفاده از mapset به یک عدد باینری تبدیل می‌کنیم و آن عدد را در یک آرایه قرار می‌دهیم که در هر دور حلقه یک عدد به انتهای این لوپ اضافه می‌شود تا در نهایت کل پیام به شکل باینری تبدیل شود. حال پس از باینری شدن پیام می‌توانیم سیگنال مورد نظر را ایجاد کنیم.

برای ایجاد سیگنال باید ابتدا یکی یکی بیت های سیگنال را به اندازه ی bit rate جدا کنیم. اگر تعداد بیت های پیام بر bit rate بخش پذیر باشد کار به سادگی انجام می‌شود. در غیر این صورت باید بیت های انتهای پیام را شیفت از راست بدهیم تا ایجاد سیگنال به درستی انجام شود.

در هر دور حلقه بیت های پیام به تعداد bit rate از پیام جدا می‌شوند. سپس این بیت ها به یک عدد دسیمال تبدیل می‌شوند تا فرکانس متناظر با آنها محاسبه شود. برای محاسبه ی فرکانس متناظر، با توجه به بیت ریت و اینکه به اندازه ی دو به توان بیت ریت حالت وجود دارد، رنج فرکانسمان را به آن تقسیم می‌کنیم. دلیل $fs/2$ هم این است که نیازی به بخش منفی سینوس نداریم. به این گونه هر فرکانس به یک بیت یا دسته بیت اختصاص پیدا میکند.

```

%%Part2 Coding function
function coded_sig = coding_freq(message,bit_rate)

    fs = 100;
    coded_sig = [];
    mapset = create_mapset();

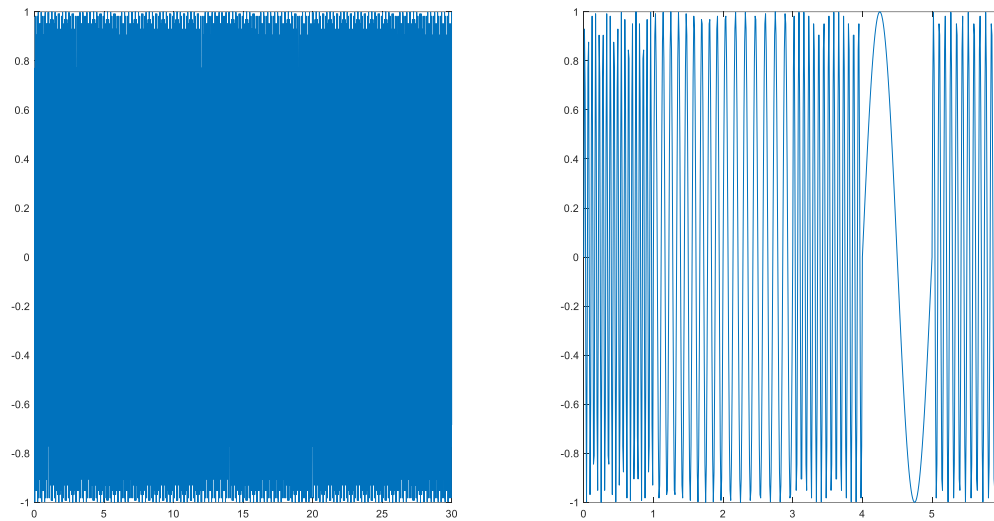
    % 1- Turning the message into a binary number
    Nch = 32;
    message_len = strlen(message);
    message_bin = cell(1,message_len);
    for i=1:message_len
        ch = message(i);
        for j=1:Nch
            if ch == mapset{1,j}
                message_bin{i} = mapset{2,j};
            end
        end
    end
    binarymessage = cell2mat(message_bin);
    binarymessage_len = length(binarymessage);

    % 2- Creating the coded signal
    f_step = floor((fs/2^bit_rate)/2);
    for z=1:bit_rate:binarymessage_len
        if z+bit_rate-1<=binarymessage_len
            bin_to_write = binarymessage(z:z+bit_rate-1);
        else
            bin_to_write = binarymessage(z:binarymessage_len);
            dec_to_write = bin2dec(bin_to_write);
            for q=length(bin_to_write):bit_rate-1
                dec_to_write = bitshift(dec_to_write,1);
            end
            bin_to_write = dec2bin(dec_to_write);
        end
        decimal = bin2dec(bin_to_write);
        frequency = f_step*decimal+ceil(f_step/2);
        t=(z/bit_rate):(1/fs):(z/bit_rate)+1-(1/fs);
        to_write = sin(2*pi*frequency*(t-(z/bit_rate)));
        coded_sig = [coded_sig to_write];
    end

end

%% 2- Coding the message
fs = 100;
message = 'signal';
bit_rate=[1,5];
figure
for i=1:2
    coded_signal = coding_freq(message,bit_rate(i));
    % 3- Plotting the result
    t = 0:(1/fs):(length(coded_signal)/fs)-(1/fs);
    subplot(1,2,i)
    plot(t,coded_signal)
end

```



تمرین ۲-۴)

برای این بخش مشابه بخش قبلی، یک سری threshold محاسبه می‌کنیم و برای هر بخش از سیگنال کدگذاری شده، ماکسیمم مقدار fft را برداشته و اندیس آن را مطابق با threshold می‌گیریم. سپس مقدار آن را به عدد دسیمال مورد نظر تبدیل کرده و در نهایت آن را باینری کرده و آن را ذخیره می‌کنیم. سپس ۵ تا ۵ تا کاراکترهای پیام باینری را جدا کرده و بررسی می‌کنیم و متناظر هر کدام در mapset را به عنوان حرف مورد نظر ذخیره کرده و در نهایت پیام استخراج شده را خروجی می‌دهیم.

```
function decoded_message = decoding_freq(coded_sig, bit_rate)
    fs = 100;
    f_step = floor((fs/(2^bit_rate))/2);
    mapset = create_mapset();
    decoded_message = [];
    % Extracting binary message
    bin_message = [];
    t = 0:(1/fs):1-(1/fs);
    thresholds = linspace(-fs/2, fs/2 - 1, fs);
    for j = 0:(length(coded_sig)/fs)-1;
        to_check = coded_sig(j*fs + 1 : (j+1)*fs);
        fftx = abs(fftshift(fft(to_check)));
        [~, idx] = max(fftx);
        detected_f = abs(thresholds(idx));
        dec = round((detected_f - ceil(f_step/2)) / f_step);
        dec = max(min(dec, 2^bit_rate - 1), 0);
        bin_message = [bin_message dec2bin(dec, bit_rate)];
    end

    % Turning the binary message into string
    for p = 1:5:length(bin_message)
        bits = bin_message(p:p+4);
        decoded_message(end+1) = mapset{1, bin2dec(bits)+1};
    end
end
```

```
% Part4 Decoding the signal
decoded_signal = cell(1,2);
for i=1:2
    decoded_signal{i} = decoding_freq(coded_signal{i},bit_rate(i));
    fprintf("bitrate = %d message = %s \n", bit_rate(i), decoded_signal{i});
end
```

Command Window

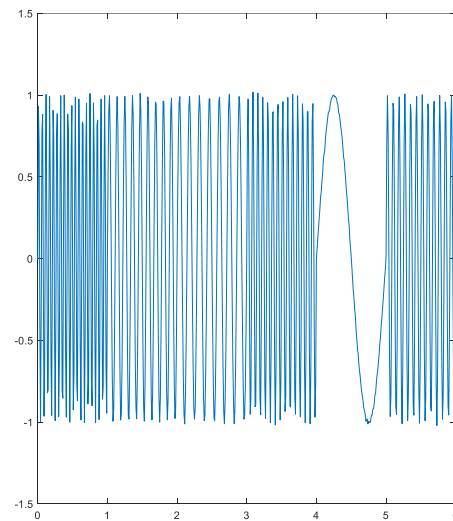
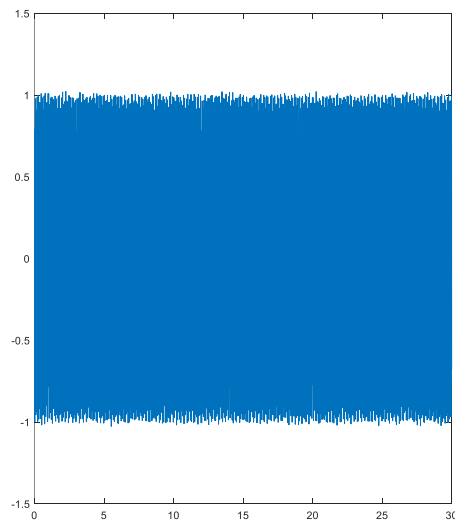
```
bitrate = 1 message = signal
bitrate = 5 message = signal
fx >> |
```

تمرین ۵-۲

در این بخش مشاهده می‌کنیم که با وجود اضافه شدن نویز با واریانس ۰/۰۰۰۱، همچنان پیام به خوبی استخراج می‌شود.

% Part5 Controlling the noise

```
m = 0.01;
noisy_signal = cell(1,2);
for i=1:2
    noise = m * randn(1, length(coded_signal{i}));
    noisy_signal{i} = coded_signal{i} + noise;
    t = 0:(1/fs):(length(noisy_signal{i})/fs)-(1/fs);
    subplot(1,2,i);
    plot(t,noisy_signal{i})
    decoded = decoding_freq(noisy_signal{i},bit_rate(i));
    fprintf("bitrate = %d message = %s \n", bit_rate(i), decoded);
end
```



Command Window

```
bitrate = 5 message = signal
bitrate = 1 message = signal
bitrate = 5 message = signal
fx >>
```


تمرین ۲-۶)

مشابه پروژه ی قبلی، هرچه سرعت بالاتر است حساسیت به نویز بیشتر است و پیام نادرست استخراج میشود. همچنین با افزایش انحراف معیار اثر نویز بر روی سیگنال بیشتر میشود. (به ازای ۳ انحراف معیار متفاوت، اثر نویز را بررسی کردیم)

% Part6 Trying different noises

M = [0.05, 1, 5];

figure

for j = 1:3

 m = M(j);

 fprintf('----- standard deviation = %f -----\n', m);

 for i=1:2

 noise = m * randn(1, length(coded_signal{i}));

 noisy_signal{i} = coded_signal{i} + noise;

 t = 0:(1/fs):(length(noisy_signal{i})/fs)-(1/fs);

 subplot(3,2,2*(j-1) + i);

 plot(t,noisy_signal{i})

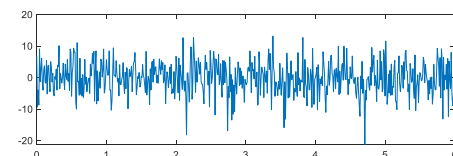
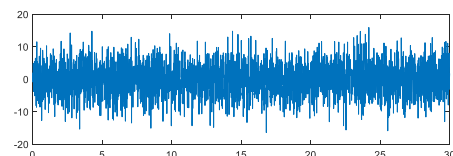
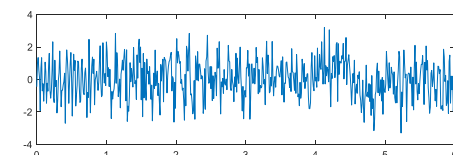
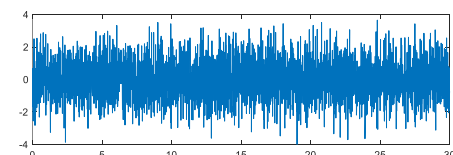
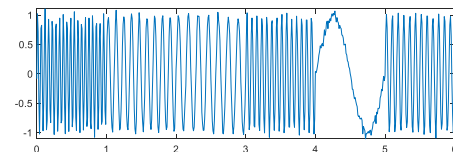
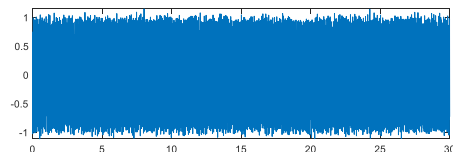
 decoded = decoding_freq(noisy_signal{i},bit_rate(i));

 fprintf("bitrate = %d message = %s \n", bit_rate(i), decoded);

 end

end

```
Command Window
----- standard deviation = 0.050000 -----
bitrate = 1 message = signal
bitrate = 5 message = signal
----- standard deviation = 1.000000 -----
bitrate = 1 message = signal
bitrate = 5 message = signal
----- standard deviation = 5.000000 -----
bitrate = 1 message = cmgxws
bitrate = 5 message = sopx",
fx >>
```



تمرین ۷-۲

مشابه پروژه ی قبلی، به علت اینکه به ازای اجرا کردن کد به علت رندوم بودن نویز در نقاط مرزی، یا پیام خودش به دست میاید یا مقداری تفاوت دارد. بنابراین ۴ بار خروجی میگیریم :

```
% Part7 Trying different noises
```

```
M = [1.4, 1.5];
```

```
figure
```

```
for i=1:2
```

```
    m = M(i);
```

```
    noise = m * randn(1, length(coded_signal{i}));
```

```
    noisy_signal{i} = coded_signal{i} + noise;
```

```
    t = 0:(1/fs):(length(noisy_signal{i})/fs)-(1/fs);
```

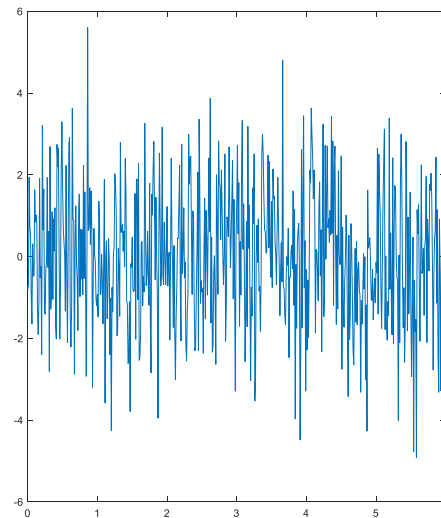
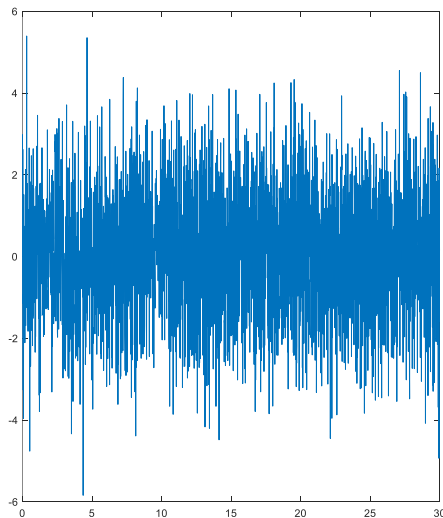
```
    subplot(1, 2, i);
```

```
    plot(t,noisy_signal{i})
```

```
    decoded = decoding_freq(noisy_signal{i},bit_rate(i));
```

```
    fprintf("bitrate = %d message = %s \n", bit_rate(i), decoded);
```

```
end
```



Command Window

```
bitrate = 1 message = signal
→ bitrate = 5 message = signal
→ bitrate = 1 message = signal
→ bitrate = 5 message = sggnal |
bitrate = 1 message = sjgnal
→ bitrate = 5 message = signal
bitrate = 1 message = signal
bitrate = 5 message = signal
fx >> |
```

تمرین ۲-۹)

اگر نرخ نمونه برداری را افزایش دهیم ، خود به خود پهنای باند نیز افزایش می یابد . میدانیم : $f = \frac{-f_s}{2} : \frac{f_s}{N} : \frac{f_s}{2} - \frac{f_s}{N}$ بنابراین با افزایش نرخ نمونه برداری ، رزولوشن فرکانسی هیچ تغییری نمی کند ، اما با توجه به رابطه بالا ، پهنای باند افزایش می یابد. یعنی ما دسترسی به پهنای باند بیشتری داریم اما با توجه به صورت تمرین ، مجاز به استفاده از تمام این پهنای باند نیستیم . بنابراین این حالت هیچ فرقی با حالت قبل ندارد و مقاومت نسبت به نویز با حالت قبل یکسان است. شایان ذکر است که اگر صورت سوال پهنای باند را محدود نمی کرد ، با افزایش نرخ نمونه برداری ، پهنای باند هم خود به خود افزایش می یافت و می توانستیم مقاومت نسبت به نویز را افزایش بدهیم. اما این اتفاق نمی افتد زیرا پهنای باند توسط صورت تمرین محدود شده است و پهنای باند مصرفی از پهنای باند قابل استفاده کمتر است.